

Concepts

Instruments

An instrument represents the specification for any tradable asset or contract. All instrument types are implemented as Rust structs that implement the `Instrument` trait. In Python, these are exposed as Cython extension types (via `nautilus_trader.model.instruments`), with parallel PyO3 representations that are converted to Cython types at the boundary. Pure Rust systems use the Rust types directly. The platform supports a range of asset classes and instrument classes:

- `Equity`: Listed shares or ETFs traded on cash markets.
- `CurrencyPair`: Spot FX or crypto pair in BASE/QUOTE format traded in cash markets.
- `Commodity`: Spot commodity instrument (e.g., gold or oil) traded in cash markets.
- `IndexInstrument`: Spot index calculated from constituents; used as a reference price and not directly tradable.
- `FuturesContract`: Deliverable futures contract with defined underlying, expiry, and multiplier.
- `FuturesSpread`: Exchange-defined multi-leg futures strategy (e.g., calendar or inter-commodity) quoted as one instrument.
- `CryptoFuture`: Dated, deliverable crypto futures contract with fixed expiry, underlying crypto, and settlement currency.
- `CryptoPerpetual`: Perpetual futures contract (perpetual swap) on crypto with no expiry; can be inverse or quanto-settled.
- `PerpetualContract`: Asset-class agnostic perpetual swap for any underlying (FX, equities, commodities, indexes, crypto).
- `OptionContract`: Exchange-traded option (put or call) on an underlying with strike and expiry.
- `OptionSpread`: Exchange-defined multi-leg options strategy (e.g., vertical, calendar, straddle) quoted as one instrument.

- `CryptoOption`: Option on a crypto underlying with crypto quote/settlement; supports inverse or quanto styles.
- `BinaryOption`: Fixed-payout option that settles to 0 or 1 based on a binary outcome.
- `Cfd`: Over-the-counter Contract for Difference that tracks an underlying and is cash-settled.
- `BettingInstrument`: Sports/gaming market selection (e.g., team or runner) tradable on betting venues.
- `SyntheticInstrument`: Synthetic instrument with prices derived from component instruments using a formula.

Symbology

All instruments should have a unique `InstrumentId`, which is made up of both the native symbol, and venue ID, separated by a period. For example, on the Binance Futures crypto exchange, the Ethereum Perpetual Futures Contract has the instrument ID `ETHUSDT-PERP.BINANCE`.

All native symbols *should* be unique for a venue (this is not always the case e.g. Binance share native symbols between spot and futures markets), and the `{symbol.venue}` combination *must* be unique for a Nautilus system.



The correct instrument must be matched to a market dataset such as ticks or order book data for logically sound operation. An incorrectly specified instrument may truncate data or otherwise produce surprising results.

Backtesting

Generic test instruments can be instantiated through the `TestInstrumentProvider`:

```
from nautilus_trader.test_kit.providers import TestInstrumentProvider

audusd = TestInstrumentProvider.default_fx_ccy("AUD/USD")
```



```
from nautilus_trader.adapters.binance.spot.providers import BinanceSpotInstrumentProvider
from nautilus_trader.model import InstrumentId
```



```

provider = BinanceSpotInstrumentProvider(client=binance_http_client)
await provider.load_all_async()

btcusdt = InstrumentId.from_str("BTCUSDT.BINANCE")
instrument = provider.find(btcusdt)

```

Or defined directly by constructing a specific instrument type:

```

from nautilus_trader.model.instruments import OptionContract

instrument = OptionContract(...) # provide all necessary parameters

```

```

use nautilus_model::instruments::CurrencyPair;
use nautilus_model::identifiers::{InstrumentId, Symbol};
use nautilus_model::types::{Currency, Price, Quantity};

let instrument = CurrencyPair::new(
    InstrumentId::from("EUR/USD.SIM"),
    Symbol::from("EUR/USD"),
    Currency::from("EUR"),
    Currency::from("USD"),
    5, // price_precision
    0, // size_precision
    Price::from("0.00001"), // price_increment
    Quantity::from("1"), // size_increment
    // ... remaining parameters
);

```

See the full instrument [API Reference](#).

Live trading

Live integration adapters have `InstrumentProvider` implementations that automatically cache the latest instrument definitions for the venue. Refer to a particular instrument by passing the matching `InstrumentId` to data and execution methods that require one.

Finding instruments

Since the same actor/strategy classes can be used for both backtest and live trading, you can get instruments in exactly the same way through the central cache:

```
from nautilus_trader.model import InstrumentId

instrument_id = InstrumentId.from_str("ETHUSD-T-PERP.BINANCE")
instrument = self.cache.instrument(instrument_id)
```

```
use nautilus_model::identifiers::InstrumentId;

let instrument_id = InstrumentId::from("ETHUSD-T-PERP.BINANCE");
let instrument = cache.instrument(&instrument_id);
```

It's also possible to subscribe to any changes to a particular instrument:

```
self.subscribe_instrument(instrument_id)
```

Or subscribe to all instrument changes for an entire venue:

```
from nautilus_trader.model import Venue

binance = Venue("BINANCE")
self.subscribe_instruments(binance)
```

When an update to the instrument(s) is received by the `DataEngine`, the object(s) will be passed to the `on_instrument()` handler. Override this method with actions to take upon receiving an instrument update:

```
from nautilus_trader.model.instruments import Instrument

def on_instrument(self, instrument: Instrument) -> None:
    # Take some action on an instrument update
    pass
```

Precision

Precision defines the number of decimal places allowed for prices and quantities on a given instrument. Every instrument specifies a `price_precision` and `size_precision` that determine the valid fractional resolution for that market.

NautilusTrader enforces precision strictly by design. This section explains the rationale and mechanics behind this approach.

Why precision is enforced

Realistic market simulation. Real exchanges only accept prices and sizes at specific precisions. A crypto spot market may support prices to 2 decimal places (e.g., `50000.01`) while a different market supports 8 (e.g., `0.00012345`). Allowing arbitrary precision in a backtest would produce fills at price levels that could never exist in production, leading to misleading performance metrics.

Venue compatibility. Most exchanges validate price and size precision on incoming orders and reject those that exceed the instrument's specification. Enforcing precision at the platform level catches a common class of these issues early. Note that venues may also enforce tick-multiple or step-size constraints beyond what the `RiskEngine` currently validates, so precision compliance alone does not guarantee venue acceptance.

Deterministic calculations. Fixed-point arithmetic with explicit precision eliminates floating-point drift and ensures calculations are reproducible across platforms and environments. Two systems processing the same data will always produce identical results.

Data integrity. The backtesting matching engine validates that all incoming market data (quotes, trades, bars) matches the instrument's declared precision. This catches mismatches between instrument definitions and data sources early, preventing silent corruption of fill prices and quantities.

How precision works

Each instrument defines two precision values:

Field	Constrains	Example
<code>price_precision</code>	Order prices, trigger prices, fills.	<code>2</code> -> <code>50000.01</code>

Field	Constraints	Example
<code>size_precision</code>	Order quantities, fill quantities.	<code>5</code> -> <code>1.00001</code>

These precisions are paired with minimum increments:

Field	Purpose
<code>price_increment</code>	Smallest valid price change (tick size).
<code>size_increment</code>	Smallest valid quantity change.

The increment's own precision must exactly match the instrument's declared precision. For example, an instrument with `price_precision=2` and `price_increment=Price(0.01, 2)` is valid, but a mismatch between these values will raise an error at instrument creation.

Where precision is enforced

Precision is validated at multiple levels throughout the platform:

1. Instrument creation: The precision of `price_increment` and `size_increment` must match `price_precision` and `size_precision` respectively.
2. Risk engine: Before an order reaches the venue, the `RiskEngine` checks that the order's price and quantity precision do not exceed the instrument's limits. Orders that fail this check are denied.
3. Matching engine: During backtesting, the matching engine validates that all incoming market data matches the instrument's precision. Mismatches raise a `RuntimeError` immediately.



The `RiskEngine` does not round values automatically. If you create a `Price` with 5 decimal places on an instrument that supports 2, the order will be denied. Use `instrument.make_price()` and `instrument.make_qty()` to round explicitly.

Working with instrument precision

Use the instrument's factory methods to create values with correct precision:

```
instrument = self.cache.instrument(instrument_id)

price = instrument.make_price(0.90500)
quantity = instrument.make_qty(150)
```



These methods round the input to the instrument's declared precision, ensuring the result will pass precision checks. Other validation rules still apply (e.g., min/max quantity limits), and `make_qty()` will raise if the rounded value is zero.



Always use `instrument.make_price()` and `instrument.make_qty()` when creating order parameters. This avoids precision mismatch errors and ensures your values have the correct number of decimal places for the instrument.

If you encounter precision mismatch errors during backtesting, verify that:

1. The instrument definition matches your data source's precision.
2. Data was not inadvertently rounded or truncated during loading.
3. Custom data loaders preserve the original precision metadata.

Limits

Certain value limits are optional for instruments and can be `None`, these are exchange dependent and can include:

- `max_quantity` (maximum quantity for a single order).
- `min_quantity` (minimum quantity for a single order).
- `max_notional` (maximum value of a single order).
- `min_notional` (minimum value of a single order).
- `max_price` (maximum valid quote or order price).
- `min_price` (minimum valid quote or order price).



Most of these limits are checked by the Nautilus `RiskEngine`, otherwise exceeding published limits *can* result in the exchange rejecting orders.

Margins and fees

Margin calculations are handled by the `MarginAccount` class. This section explains how margins work and introduces key concepts you need to know.

When do margins apply?

Each exchange (e.g., CME or Binance) operates with a specific account type that determines whether margin calculations are applicable. When setting up an exchange venue, you'll specify one of these account types:

- `AccountType.MARGIN`: Accounts that use margin calculations, which are explained below.
- `AccountType.CASH`: Simple accounts where margin calculations do not apply.
- `AccountType.BETTING`: Accounts designed for betting, which also do not involve margin calculations.

Vocabulary

To understand trading on margin, let's start with some key terms:

Notional Value: The total contract value in the quote currency. It represents the full market value of your position. For example, with EUR/USD futures on CME (symbol 6E).

- Each contract represents 125,000 EUR (EUR is base currency, USD is quote currency).
- If the current market price is 1.1000, the notional value equals $125,000 \text{ EUR} \times 1.1000$ (price of EUR/USD) = 137,500 USD.

Leverage (`leverage`): The ratio that determines how much market exposure you can control relative to your account deposit. For example, with 10× leverage, you can control 10,000 USD worth of positions with 1,000 USD in your account.

Initial Margin (`margin_init`): The margin rate required to open a position. It represents the minimum amount of funds that must be available in your account to open new positions. This is only a pre-check; no funds are actually locked.

Maintenance Margin (`margin_maint`): The margin rate required to keep a position open. This amount is locked in your account to maintain the position. It is always lower than the initial margin. You can view the total blocked funds (sum of maintenance margins for open positions) using the following in your strategy:

```
self.portfolio.balances_locked(venue)
```



Maker/Taker Fees: The fees charged by exchanges based on your order's interaction with the market:

- **Maker Fee** (`maker_fee`): A fee (typically lower) charged when you "make" liquidity by placing an order that remains on the order book. For example, a limit buy order below the current price adds liquidity, and the *maker* fee applies when it fills.
- **Taker Fee** (`taker_fee`): A fee (typically higher) charged when you "take" liquidity by placing an order that executes immediately. For instance, a market buy order or a limit buy above the current price removes liquidity, and the *taker* fee applies.

Fee rate sign convention: Nautilus uses a consistent sign convention for fee rates across all adapters and the backtesting engine:

- Positive fee rate = commission (fee charged, reducing account balance).
- Negative fee rate = rebate (fee earned, increasing account balance).

For example, a maker fee of `-0.00025` means you receive a 0.025% rebate for providing liquidity, while a taker fee of `0.00075` means you pay a 0.075% commission for taking liquidity.



Different exchanges use different sign conventions in their APIs. Nautilus adapters normalize these to the convention above. If you're manually specifying fee rates for backtesting, ensure you follow this convention.



Not all exchanges or instruments implement maker/taker fees. If absent, set both `maker_fee` and `taker_fee` to 0 for the `Instrument` (e.g., `FuturesContract`, `Equity`, `CurrencyPair`, `Commodity`, `Cfd`),

```
BinaryOption, BettingInstrument)).
```

Margin calculation formula

The `MarginAccount` class calculates margins using the following formulas:

```
# Initial margin calculation
margin_init = (notional_value / leverage * margin_init) + (notional_value / leverage * taker_fee)

# Maintenance margin calculation
margin_maint = (notional_value / leverage * margin_maint) + (notional_value / leverage * taker_fee)
```

Key Points:

- Both formulas follow the same structure but use their respective margin rates (`margin_init` and `margin_maint`).
- Each formula consists of two parts:
 - Primary margin calculation: Based on notional value, leverage, and margin rate.
 - Fee Adjustment: Accounts for the maker/taker fee.

Implementation details

For those interested in exploring the technical implementation:

- [nautilus_trader/accounting/accounts/margin.pyx](#)
- Key methods: `calculate_margin_init(self, ...)` and `calculate_margin_maint(self, ...)`

Commissions

Trading commissions represent the fees charged by exchanges or brokers for executing trades. While maker/taker fees are common in cryptocurrency markets, traditional exchanges like CME often employ other fee structures, such as per-contract commissions. NautilusTrader supports multiple commission models to accommodate diverse fee structures across different markets.

Built-in fee models

The framework provides two built-in fee model implementations:

1. `MakerTakerFeeModel`: Implements the maker/taker fee structure common in cryptocurrency exchanges, where fees are calculated as a percentage of the trade value.
2. `FixedFeeModel`: Applies a fixed commission per trade, regardless of the trade size.

Creating custom fee models

While the built-in fee models cover common scenarios, you might encounter situations requiring specific commission structures. NautilusTrader's flexible architecture allows you to implement custom fee models by inheriting from `FeeModel`.

For example, if you're trading futures on exchanges that charge per-contract commissions (like CME), you can implement a custom fee model. When creating custom fee models, we inherit from the `FeeModel` base class, which is implemented in Cython for performance reasons. This Cython implementation is reflected in the parameter naming convention, where type information is incorporated into parameter names using underscores (like `Order_order` or `Quantity_fill_qty`).

While these parameter names might look unusual to Python developers, they're a result of Cython's type system and help maintain consistency with the framework's core components. Here's how you could create a per-contract commission model:

```
class PerContractFeeModel(FeeModel):
    def __init__(self, commission: Money):
        super().__init__()
        self.commission = commission

    def get_commission(self, Order_order, Quantity_fill_qty, Price_fill_px, Instrument_instrument):
        total_commission = Money(self.commission * Quantity_fill_qty, self.commission.currency)
        return total_commission
```

This custom implementation calculates the total commission by multiplying a `fixed per-contract fee` by the `number of contracts` traded. The `get_commission(...)` method receives information about the order, fill quantity, fill price and instrument, allowing for flexible commission calculations based on these parameters.

Our new class `PerContractFeeModel` inherits class `FeeModel`, which is implemented in Cython, so notice the Cython-style parameter names in the method signature:

- `Order_order`: The order object, with type prefix `Order_`.
- `Quantity_fill_qty`: The fill quantity, with type prefix `Quantity_`.
- `Price_fill_px`: The fill price, with type prefix `Price_`.
- `Instrument_instrument`: The instrument object, with type prefix `Instrument_`.

These parameter names follow NautilusTrader's Cython naming conventions, where the prefix indicates the expected type. While this might seem verbose compared to typical Python naming conventions, it ensures type safety and consistency with the framework's Cython codebase.

Using fee models in practice

To use any fee model in your trading system, whether built-in or custom, you specify it when setting up the venue. Here's an example using the custom per-contract fee model:

```
from nautilus_trader.model.currencies import USD
from nautilus_trader.model.objects import Money, Currency

engine.add_venue(
    venue=venue,
    oms_type=OmsType.NETTING,
    account_type=AccountType.MARGIN,
    base_currency=USD,
    fee_model=PerContractFeeModel(Money(2.50, USD)), # 2.50 USD per contract
    starting_balances=[Money(1_000_000, USD)], # Starting with 1,000,000 USD balance
)
```



When implementing custom fee models, ensure they accurately reflect the fee structure of your target exchange. Even small discrepancies in commission calculations can significantly impact strategy performance metrics during backtesting.

Additional info

The raw instrument definition as provided by the exchange (typically from JSON serialized data) is also included as a generic Python dictionary. This is to retain all information which is not

necessarily part of the unified Nautilus API, and is available to the user at runtime by calling the `.info` property.

Synthetic instruments

The platform supports creating customized synthetic instruments, which can generate synthetic quote and trades. These are useful for:

- Enabling `Actor` and `Strategy` components to subscribe to quote or trade feeds.
- Triggering emulated orders.
- Constructing bars from synthetic quotes or trades.

Synthetic instruments cannot be traded directly, as they are constructs that only exist locally within the platform. They serve as analytical tools, providing useful metrics based on their component instruments.

In the future, we plan to support order management for synthetic instruments, enabling trading of their component instruments based on the synthetic instrument's behavior.



The venue for a synthetic instrument is always designated as `'SYNTH'`.

A synthetic instrument derives its price from a formula over two or more component instruments. It behaves like a normal instrument for subscriptions and emulation triggers, but it exists only within NautilusTrader.

See the [Synthetics](#) guide for:

- The formula language reference.
- Supported operators and functions.
- Creation and update examples.
- Triggering emulated orders from synthetic prices.
- Validation rules and error handling.

Related guides

- [Data](#) - Market data types for instruments.
- [Orders](#) - Orders reference instruments.

[< Strategies](#)[Previous Page](#)[Continuous Futures >](#)[Next Page](#)